

Corso di Laurea in Informatica A.A. 2024-2025

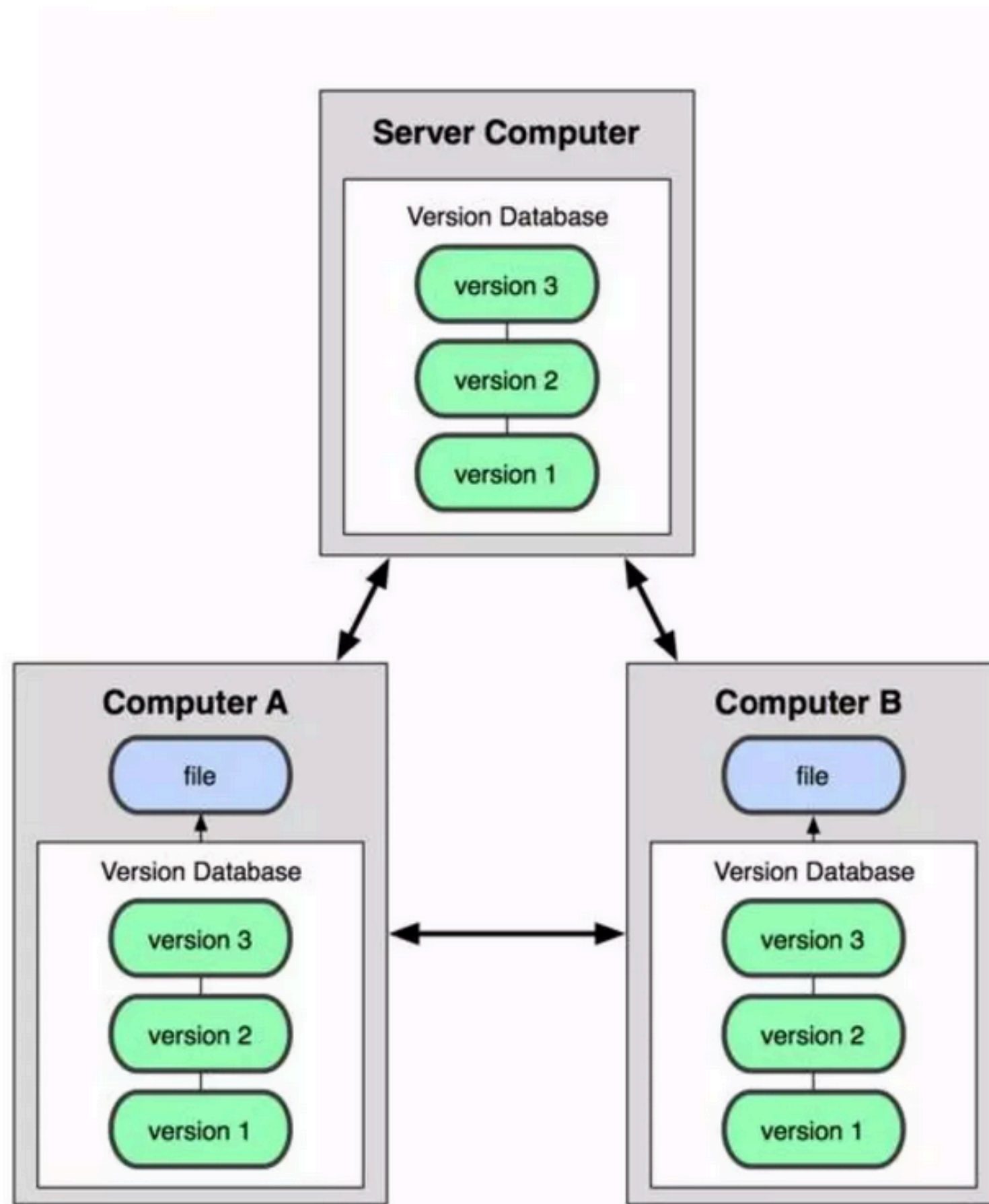
Laboratorio di Sistemi Operativi

Alessandra Rossi

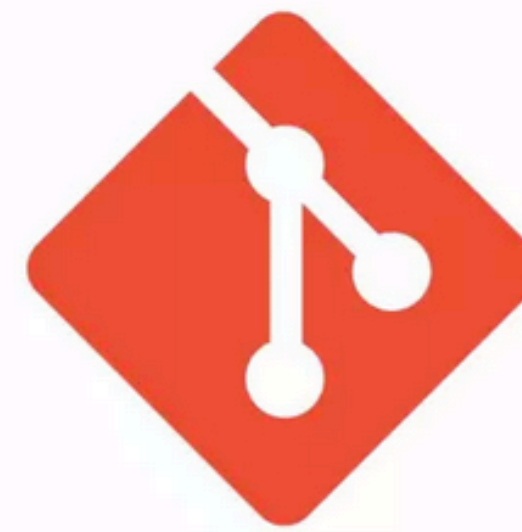
GIT

E' un distributed version control system

GIT



GIT = sistema di Controllo di Versione
Distribuiti DVCS



git



GitLab



GIT

Git considera i propri dati come una serie di istantanee (snapshot).

Ogni volta che modifichi un file lo stato del tuo progetto in Git verrà aggiornato.

Git fondamentalemente fa un'immagine di tutti i file in quel momento, salvando un riferimento allo **snapshot**.

Per essere efficiente, se alcuni file non sono cambiati, Git non li risalva, ma ***crea semplicemente un collegamento*** al file precedente già salvato.

PRO: velocità nel riassembleare la storia a fronte di un'occupazione di spazio non ottimizzata

CONTRO: maggiore occupazione di spazio rispetto ad altre soluzioni

GIT

Git garantisce la storia dei suoi file per ogni distribuzione del progetto.

Questo meccanismo che garantisce facilità nel recupero della storia dai client che utilizzano il progetto in caso di guasti al server principale.

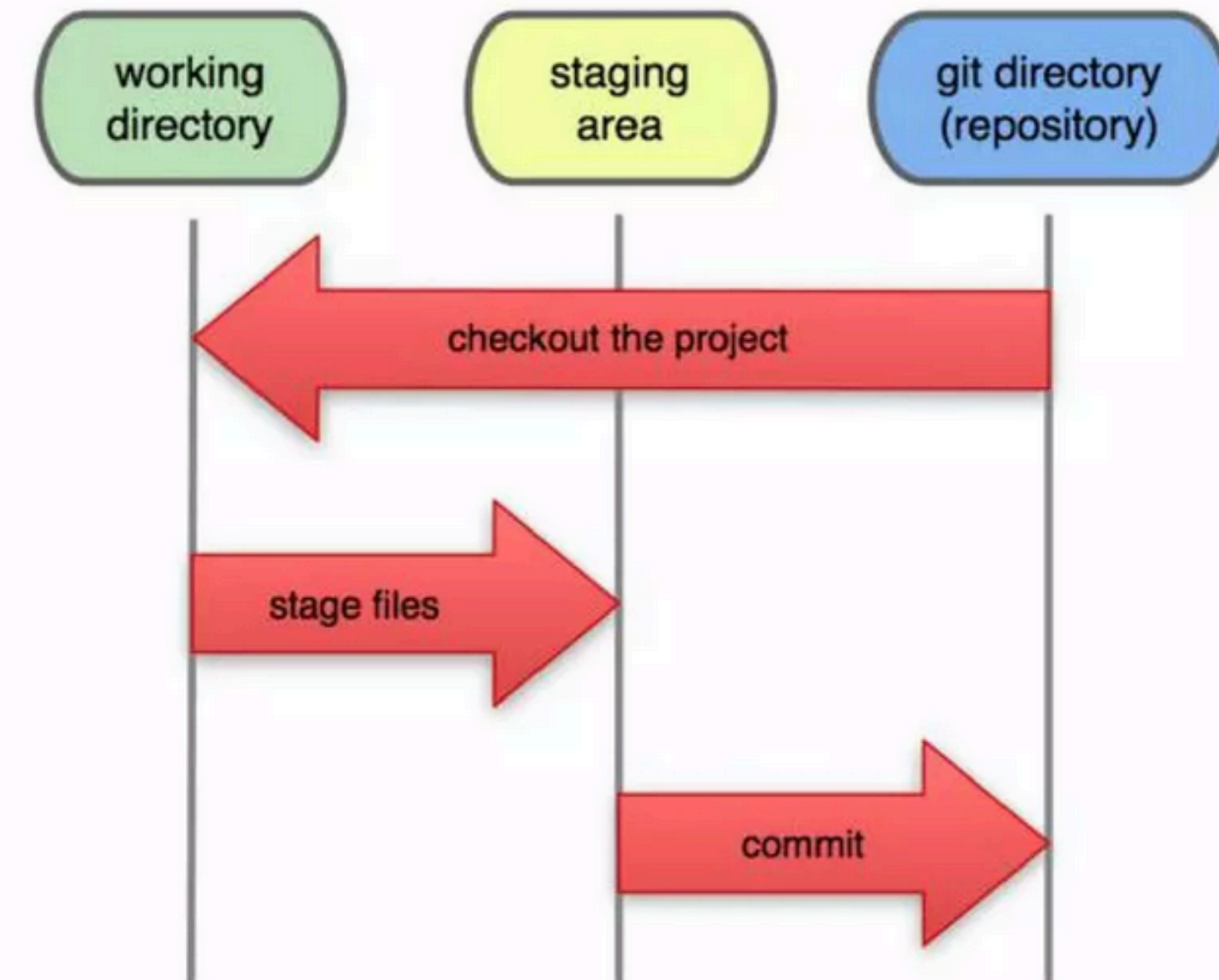
Ogni file in un progetto git è controllato tramite checksum-> tracciamento basato sul contenuto del file. Ogni cambiamento effettuato al contenuto del file è soggetto al controllo di Git.

Git usa l'algoritmo SHA-1 a caratteri.

In Git i file possono trovarsi in 3 stati differenti:

- **modified**
- **staged**
- **committed**

I file all'interno di un progetto possono essere **tracked** o **untracked**



GIT

Il comando 'git config' che ti permetterà d'impostare e conoscere le variabili di configurazione.

Esse possono essere:

- **di sistema** se si usa il parametro `--system` varranno per tutti gli utenti e tutti i repository
- **per utente** se si usa il parametro `--global` varranno solo per l'utente in uso e per tutti i suoi repository
- **per repository** se non si usa alcun parametro, varranno solo per il repository corrente.

Ogni livello vince sul livello precedente, così che i valori di progetto hanno più importanza rispetto a quelli utente che hanno più importanza rispetto a quelli di sistema.

GIT

Puoi creare un nuovo repository (progetto in GIT) in due modi:

- convertire una cartella di progetto esistente in un progetto GIT (\$ **git init**)
- clonare un progetto esistente (\$ **git clone [url]**)

Una volta iniziato un progetto potrai tracciare i tuoi file semplicemente aggiungendoli a quelli **tracked**:

\$ **git add .** -> aggiunge tutti i file nella cartella e sottocartella tra quelli tracciati

\$ **git add nomeFile.txt** -> aggiunge il file nomeFile.txt tra quelli tracciati

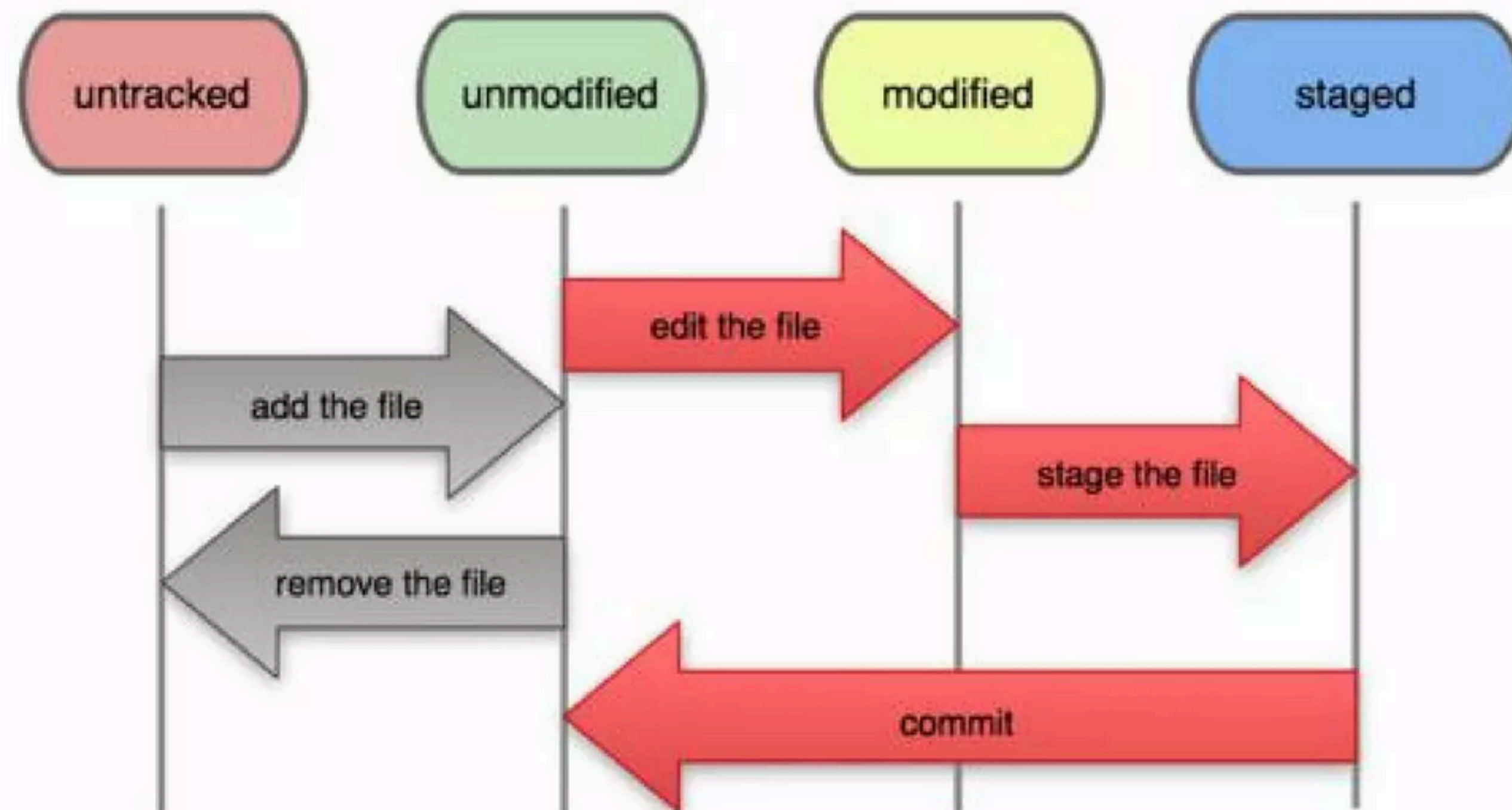
\$ **git clone (git|http|https)://url/[nomeProgetto].git PrjDemo**-> Scarica e clona tutto il progetto git dall'url nella cartella di progetto PrjDemo

Quando esegui **git clone** vengono scaricate tutte le versioni di ciascun file della cronologia del progetto. Infatti, se si danneggiasse il disco del tuo server, potresti usare qualsiasi clone di qualsiasi client per ripristinare il server allo stato in cui era quando è stato clonato.

GIT

Quando editi dei file, Git li vede come **modified**, perché sono cambiati rispetto all'ultimo **commit**.

Metti nell'area di **stage** i file modificati, poi fai la **commit** di tutto ciò che è in quest'area, e quindi il ciclo si ripete.



GIT

```
$ git status -> ritorna informazioni sui file tracciati/non tracciati, modificati e sulla branch in uso  
$ git add es1.txt -> aggiunge il file es1.txt nell'area di stage all'istante di esecuzione del comando  
$ git commit -m "Fix table error example" -> committa la staging area e ne fa lo snapshot
```

GIT

GIT mette a disposizione un file **.gitignore** che permette di ignorare file e cartelle di cui non vuoi tener traccia ma che risiedono per loro stessa natura all'interno del progetto. Tali file possono essere ad esempio i file temporanei del tuo sistema operativo o file generati dalla compilazione del progetto (che quindi si potranno nuovamente rigenerare in futuro) o altro ancora.

Per generare facilmente un file .gitignore (un semplice file di testo) visita

<https://www.gitignore.io/>

Altre informazioni

<https://git-scm.com/book/it/v1/Basi-di-Git-Salvare-le-modifiche-sul-repository#Ignorare-File>

GIT

Per vedere cosa hai modificato, ma non ancora nello stage, digita **git diff** senza altri argomenti, questo comando confronta cosa c'è nella tua directory di lavoro con quello che c'è nella tua area di stage.

Se vuoi vedere cosa c'è nello stage e che farà parte della prossima commit, puoi usare **git diff --cached** oppure **git diff --staged**.

È importante notare che **git diff** di per sé non visualizza tutte le modifiche fatte dall'ultima commit, ma solo quelle che non sono ancora in stage. Questo può confondere, perché se hai messo in stage tutte le tue modifiche, git diff non mostrerà nulla.

\$ **git diff**

Il risultato mostra le tue modifiche che ancora non hai messo nello stage.

CA: Amministratore: Prompt dei comandi

```
C:\Users\Valerio\git\gitRepoExample>git diff --stage
diff --git a/es1.txt b/es1.txt
index 8477a6c..a4b017c 100644
--- a/es1.txt
+++ b/es1.txt
@@ -1,1 @@
-Originale1fhfghdf
\ No newline at end of file
+Originale1fhfghdf

C:\Users\Valerio\git\gitRepoExample>
```


GIT

\$ **git log** -> mostra una serie di informazioni sui commit passati, dalle più recenti a ritroso

Amministratore: Prompt dei comandi - git log

```
C:\Users\Valerio\git\gitRepoExample>git log
commit 6457ee1fe1bb38e68ef90864d35a8bd915a8c2fe
Author: Valerio <valix85@gmail.com>
Date:   Mon Mar 6 17:17:10 2017 +0100

    Commit demo

commit 1553736ab5e130ce1e80ea921608a06e53034ade
Author: Valerio <valix85@gmail.com>
Date:   Fri Feb 24 16:21:09 2017 +0100

    numerazione allungata

commit 8f2cf80b3d6ef007c790b61a6c9f57ebd5528514
Author: Valerio <valix85@gmail.com>
Date:   Fri Feb 24 16:12:17 2017 +0100

    modificato numeri
```

Tra le informazioni si trovano:

il **checksum** che identifica il commit

l'**autore** del commit (`git config user.name`)

la **data** e l'**ora** di avvenuto commit

la **descrizione** inserita al momento del commit

Se avviato con il parametro `-p` mostra il diff dei file, se si passa anche un numero N limita agli ultimi N commit la visualizzazione, con il parametro `--word-diff` vengono evidenziate le sole parole modificate.

I simboli “+” e “-” mostrano ciò che è stato aggiunto e rimosso.

GIT

```
$ git reset HEAD nomeFile.ext -> rimuove dalla staging area un file inserito, non ne resetta le modifiche
```

```
$ git checkout -- nomeFile.ext -> rimuove dalla staging area un file inserito e lo riporta allo stato dell'ultimo commit, resetta le modifiche!
```

```
$ git commit --amend -> ripristina l'ultimo commit dando la possibilità di cambiare il commento, se presenti in staging area gli stessi file ma aggiornati nuovamente essi verranno caricati con le ultime modifiche ( non perdo le ultime modifiche fatte, vengono “aggiornati” ). Concettualmente permette di fondere ciò che ho nello staging con l'ultimo mio commit effettuato, aggiornando quindi l'ultimo commit con i file di cui mi ero scordato.
```

```
$ git rebase -i <num_commit> : è utile per fare il pick '&' squash di tutto un pezzo di ramo e accorpare le modifiche in un unico nuovo commit (posso lavorare in locale con centinaia di commit per risolvere un problema e caricare un solo commit al remoto )
```


GIT

\$ **git remote -v** -> mostra l'elenco dei repository remoti (-v con url) associati al progetto in uso

\$ **git remote add <nomeUrlRemoto> <URL>** -> aggiunge un repository remoto che si chiama nomeUrlRemoto e che punta all'indirizzo URL

\$ **git remote remove <nomeUrlRemoto>** -> rimuove l'indirizzo dichiarato col nome nomeUrlRemoto dal progetto

```
C:\Users\Valerio\git\gitRepoExample>git remote remove rem1  
  
C:\Users\Valerio\git\gitRepoExample>git remote add remoteDemoUrl git://...indirizzoRemoto.git  
  
C:\Users\Valerio\git\gitRepoExample>git remote -v  
remoteDemoUrl  git://...indirizzoRemoto.git (fetch)  
remoteDemoUrl  git://...indirizzoRemoto.git (push)
```

\$ **git remote rename <nomeUrl> <nuovoNomeUrl>** -> rinomina il riferimento locale di un repository remoto

\$ **git remote show <nomeUrlRemoto>** -> mostra informazioni sul repository remoto, sulle attuali brach derivanti da quel repository e sulla predefinita che verrà utilizzata in fase di push

Il repository con il nome “origin” è il repository predefinito remoto (si crea automaticamente quando si clona un progetto), master è il nome predefinito per quello locale.

GIT

\$ **git fetch** <nomeUrlRemoto> -> recupera tutta la storia dal repository remoto, non li unisce al mio repository

\$ **git pull** <nomeUrlRemoto> -> recupera tutto il contenuto dal repository remoto e lo unisce al mio repository

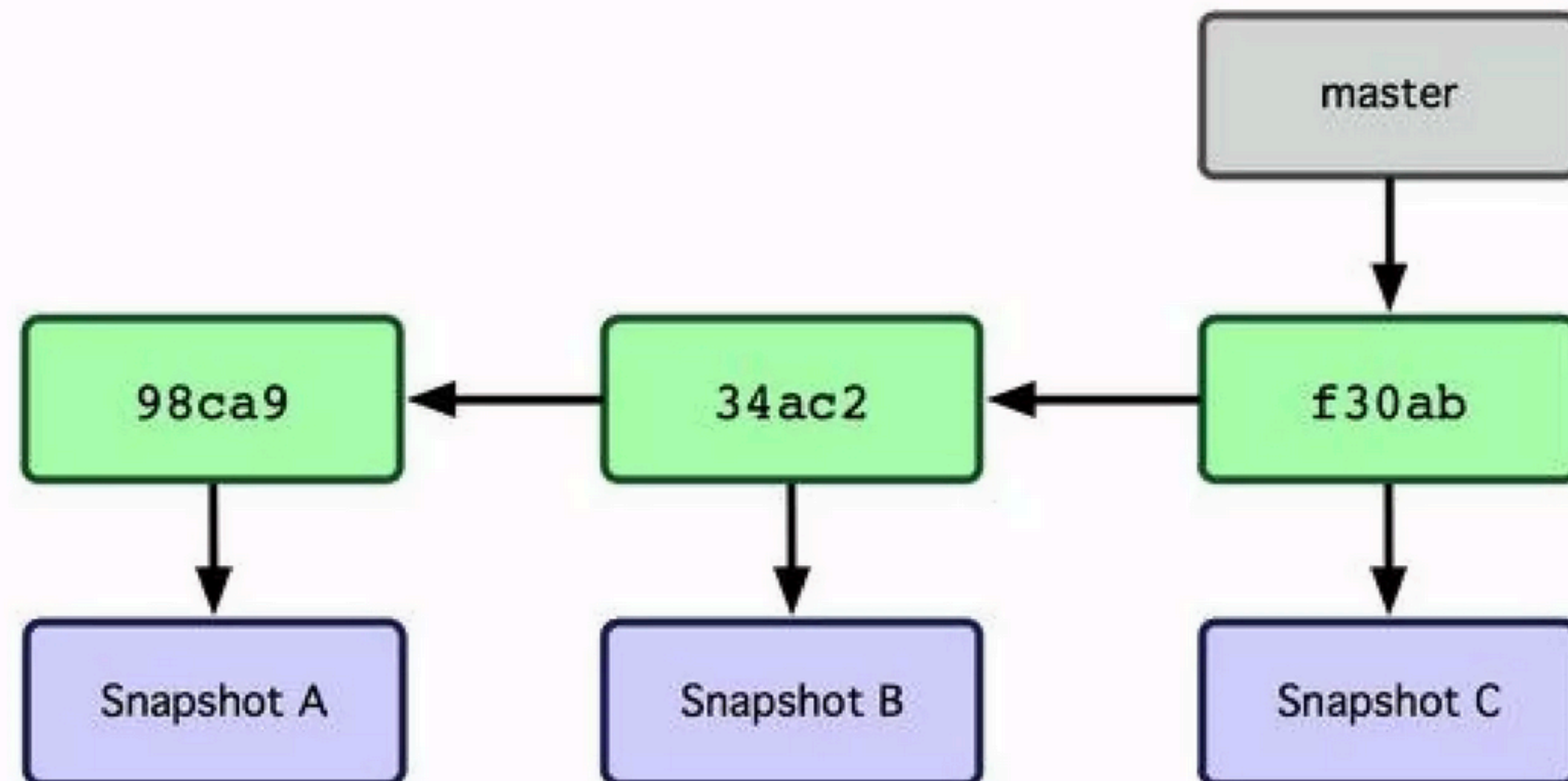
\$ **git push** <nomeUrlRemoto> <branch> -> carica il contenuto del tuo repository (<branch>) sulla destinazione remota. Fallirà se la tua versione non sarà l'ultima, in tal caso dovrai scaricare e unire il contenuto remoto col tuo e poi caricarlo.

GIT

In Git un **branch** (ramo) è semplicemente un puntatore ad uno di questi commit.

Il nome del ramo principale in Git è **master**.

Quando inizi a fare dei commit stai spostando il ramo master facendolo puntare all'ultimo commit che hai eseguito. Ogni volta che invierai un commit, lui si sposterà in avanti automaticamente.



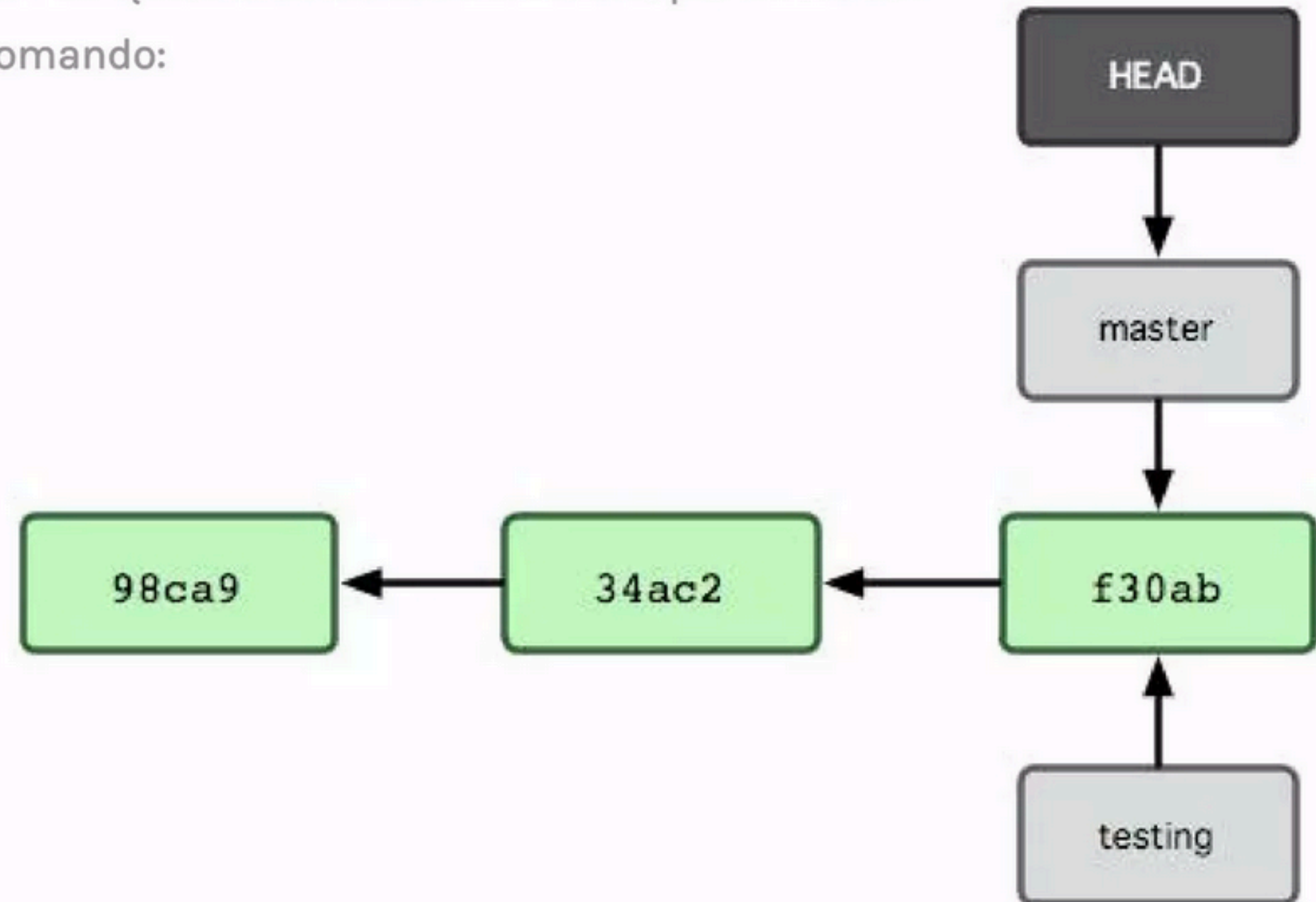
GIT

In Git puoi creare facilmente un nuovo branch con il comando

\$ **git branch <nomeNuovoBranch>** -> si crea un nuovo puntatore al commit attuale, non sposta il puntatore di HEAD.

Il puntatore HEAD serve a informare GIT su quale branch deve lavorare, per cambiare il ramo su cui lavorare devi usare il comando:

\$ **git checkout <nomeBranch>**



GIT - Esempio

Molti GIT effettuano l'autenticazione tramite SSH Public Key

```
$ cd ~/.ssh
$ ls
authorized_keys2  id_dsa          known_hosts
config           id_dsa.pub
```

```
$ ssh-keygen -o
Generating public/private rsa key pair.
Enter file in which to save the key (/home/schacon/.ssh/id_rsa):
Created directory '/home/schacon/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/schacon/.ssh/id_rsa.
Your public key has been saved in /home/schacon/.ssh/id_rsa.pub.
The key fingerprint is:
d0:82:24:8e:d7:f1:bb:9b:33:53:96:93:49:da:9b:e3 schacon@mylaptop.local
```


Git - Esempio

```
$ cat ~/.ssh/id_rsa.pub  
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAKl0UpkDHrfHY17SbrmTIpNLTGK9Tjom/BWDSU  
GPl+nafzlHDTYW7hdI4yZ5ew18JH4JW9jbhUFrviQzM7xlELEVf4h9lFX5QVkbPppSwg0cda3  
Pbv7k0dJ/MTyBlWXFCR+HAo3FXRitBqxiX1nKhXpHAZsMciLq8V6RjsNAQwdsdMFvSlVK/7XA  
t3FaoJoAsncM1Q9x5+3V0Ww68/eIFmb1zuUFljQJKprX88XypNDvjYNby6vw/Pb0rwert/En  
mZ+AW40ZPnTPI89ZPmVMLuayrD2cE86Z/il8b+gw3r3+1nKatmIkjn2so1d01QraTlMqVSsbx  
NrRFi9wrf+M7Q== schacon@mylaptop.local
```

GIT

```
[~/temp:alexandra]$ git clone git@gitlab.com:alexarossi/lso-esercizi.git
Cloning into 'lso-esercizi'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100%(6/6), done.
remote: Compressing objects: 100%(3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100%(6/6), done.
```

Find file

Web IDE

↓

↓

Clone ↓

+

Clone with SSH

git@gitlab.com:alexarossi/lso-e

Clone with HTTPS

https://gitlab.com/alexarossi/l

Open in your IDE

Visual Studio Code (SSH)

Visual Studio Code (HTTPS)

IntelliJ IDEA (SSH)

IntelliJ IDEA (HTTPS)

GIT

```
[[ ~/temp/ISO-esercizi : al handra]$ ls
README.md
[[ ~/temp/ISO-esercizi : al handra]$ cat README.md
# ISO Esercizi

Da usare per testing
```

```
[[ ~/temp/ISO-esercizi : al handra]$ git checkout -b testbranch
Switched to a new branch 'testbranch'
[[ ~/temp/ISO-esercizi : al handra]$
```

GIT

```
[~/temp/ISO-esercizi:al handra]$ git status
On branch testbranch
nothing to commit, working tree clean
[~/temp/ISO-esercizi:al handra]$
```

```
[~/temp/ISO-esercizi:al handra]$ echo "Hello world\!" >> README.md
[~/temp/ISO-esercizi:al handra]$ cat README.md
# ISO Esercizi
Da usare per testing
Hello world\!
[~/temp/ISO-esercizi:al handra]$
```


GIT

```
[~/temp/iso-esercizi:al handra]$ git status
On branch testbranch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
[~/temp/iso-esercizi:al handra]$
```

GIT

```
[~/temp/iso-esercizi:al handra]$ git add README.md
[~/temp/iso-esercizi:al handra]$ git commit -m "I added a new line in the readme for welcoming people"
[testbranch e9e0d69] I added a new line in the readme for welcoming people
1 file changed, 1 insertion(+)
[~/temp/iso-esercizi:al handra]$ git status
On branch testbranch
nothing to commit, working tree clean
```


GIT

```
[~/temp/iso-esercizi:al handra]$ git push
fatal: The current branch testbranch has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin testbranch

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

[~/temp/iso-esercizi:al handra]$ git push --set-upstream origin testbranch
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 334 bytes | 334.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for testbranch, visit:
remote:   https://gitlab.com/al exarossi /iso-esercizi /-/merge_requests/new?merge_request%5Bsource_branch%5D=testbranch
remote:
To gitlab.com al exarossi /iso-esercizi .git
 * [new branch]      testbranch -> testbranch
branch 'testbranch' set up to track 'origin/testbranch'.
```

GIT

Alessandra > LSO Esercizi > Repository > **Branches**

Overview Active Stale All

Filter by branch name



Delete merged branches

New branch

Active branches

 testbranch 				
 e9e0d693 · I added a new line in the readme for welcoming people · 5 minutes ago	0	1	Merge request	Compare
 main  default protected				
 6bb4c56d · Update README.md · 29 minutes ago				

GIT

Alessandra > LSO Esercizi > Merge requests > !1

Draft: Resolve "New issue example"

Edit

Code ▾



 Open Alessandra requested to merge `1-new-issue-example` into `main` just now

Overview 0

Commits 0

Pipelines 0

Changes 0

Closes #1



0



0



This merge request contains no changes.

Use merge requests to propose changes to your project and discuss them with your team. To make changes, push a commit or edit this merge request to use a different branch. With [CI/CD](#), automatically test your changes before merging.

Create file



Activity

Sort or filter ▾



Alessandra added `Doing` label just now

GIT

```
[~/temp/ISO-esercizi:al handra]$ git fetch
From gitlab.com:alexarossi/ISO-esercizi
* [new branch]      1-new-issue-example -> origin/1-new-issue-example
[~/temp/ISO-esercizi:al handra]$ git pull
Already up to date.
```

```
[~/temp/ISO-esercizi:al handra]$ git checkout 1-new-issue-example
branch '1-new-issue-example' set up to track 'origin/1-new-issue-example'.
Switched to a new branch '1-new-issue-example'
```


GIT

```
[~/temp/iso-esercizi:al handra]$ git commit echo "Let's add a new line for testing" >> README.md
[~/temp/iso-esercizi:al handra]$ git status
On branch 1-new-issue-example
Your branch is up to date with 'origin/1-new-issue-example'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
[~/temp/iso-esercizi:al handra]$ git add README.md
[~/temp/iso-esercizi:al handra]$ git commit
[1-new-issue-example 6d64568] new line for testing purposes
 1 file changed, 1 insertion(+)
[~/temp/iso-esercizi:al handra]$ git status
On branch 1-new-issue-example
Your branch is ahead of 'origin/1-new-issue-example' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```


GIT

```
[~/temp/iso-esercizi:al handra]$ git push
Enumerating objects: 5, done.
Counting objects: 100%(5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100%(2/2), done.
Writing objects: 100%(3/3), 331 bytes | 331.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: View merge request for 1-new-issue-example:
remote: https://gitlab.com/al exarossi /iso-esercizi /-/merge_requests/1
remote:
To gitlab.com al exarossi /iso-esercizi .git
   6bb4c56..6d64568 1-new-issue-example -> 1-new-issue-example
[~/temp/iso-esercizi:al handra]$
```


Draft: Resolve "New issue example"

Edit Code ⌵ ⋮

 Open Alessandra requested to merge `1-new-issue-example` into `main` 1 minute ago

Overview 0 Commits 0 Pipelines 0 Changes 0

Closes #1

 0  0 

 Approval is optional

 Merge blocked: merge request must be marked as ready. It's still marked as draft.

Mark as ready

Merge details

- 1 commit and 1 merge commit will be added to `main`.
- Source branch will be deleted.
- Mentions issue [#1](#)

Activity

Sort or filter ⌵

-  Alessandra added Doing label 1 minute ago
-  Alessandra added 1 commit just now
 - [6d645686](#) - new line for testing purposes[Compare with previous version](#)

Draft: Resolve "New issue example"

Edit Code ⋮

 Merged Alessandra requested to merge [1-new-issue-example](#) into [main](#) 1 minute ago

Overview 0 Commits 0 Pipelines 0 Changes 0

Closes [#1](#)

 0  0 

 Approval is optional

 Merged by  [Alessandra](#) just now

Revert Cherry-pick

Merge details

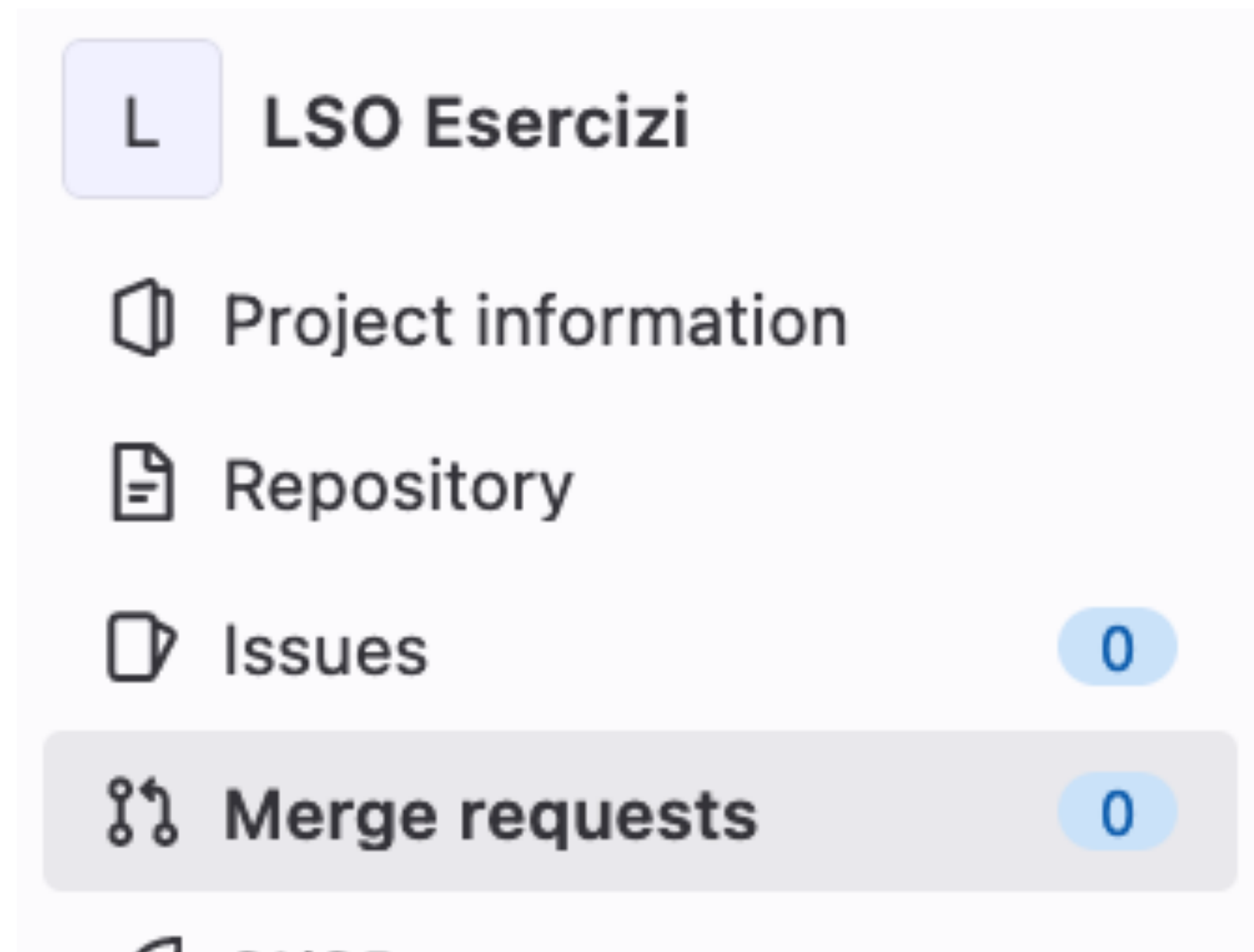
- Changes merged into [main](#) with [2c1ea7dc](#).
- Deleted the source branch.
- Mentions issue [#1](#)

Activity

Sort or filter ▾

-  Alessandra added [Doing](#) label 1 minute ago
-  Alessandra added 1 commit just now
 - [6d645686](#) - new line for testing purposes[Compare with previous version](#)
-  Alessandra marked this merge request as **ready** just now
-  Alessandra mentioned in commit [2c1ea7dc](#) just now
-  Alessandra merged just now

GIT



GIT

```
[~/temp/iso-esercizi:alhandra]$ git push
To gitlab.com:alexarossi/iso-esercizi.git
! [rejected]        testbranch -> testbranch (fetch first)
error: failed to push some refs to 'gitlab.com:alexarossi/iso-esercizi.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
~/temp/iso-esercizi:alhandra$
```


GIT

```
[~/temp/lso-esercizi:al handra]$ git fetch
[~/temp/lso-esercizi:al handra]$ git pull
error: Pulling is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
[~/temp/lso-esercizi:al handra]$ git diff
diff --cc README.md
index 1958369,6c78ff3..0000000
--- a/README.md
+++ b/README.md
@@@ -1,5 -1,9 +1,14 @@@
    # LSO Esercizi

    Da usare per testing
++<<<<<< HEAD
+Hello world\!
+adding conflict
++=====
+Hello world!
+
+Add something else here!
+
+And something more
+
++>>>>>> 251cdb11b7db35b50f a3388f e725cdf 0621bd314
```

GIT

- Eliminare le differenze
- Effettuare lo stage, poi pull e infine ripetere
- Creare un nuovo commit
- Effettuare il merge

```
git mergetool
```

GIT

1. git log --merge

The git log --merge command helps to produce the list of commits that are causing the conflict

2. git diff

The git diff command helps to identify the differences between the states repositories or files

3. git checkout

The git checkout command is used to undo the changes made to the file, or for changing branches

4. git reset --mixed

The git reset --mixed command is used to undo changes to the working directory and staging area

5. git merge --abort

The git merge --abort command helps in exiting the merge process and returning back to the state before the merging began

6. git reset

The git reset command is used at the time of merge conflict to reset the conflicted files to their original state

GIT

```
git cherry-pick commitSha
```

GIT

`https://www.atlassian.com/git/tutorials/`



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

